

Beyond Accuracy: Making Better Classification Decisions

Dr. Marcelo Feighelstein

The Problem with a Fixed Threshold

A classifier doesn't just say **Yes** or **No** — it estimates a **probability**. The final prediction depends on a threshold you choose. Should it always be 0.5?

🫀 Patient A

Heart disease probability = **0.72**

🤔 The Question

At threshold 0.5 → Disease. At threshold 0.8 → Healthy. Same model, different outcome.

🎯 The Challenge

Choosing the right threshold can mean the difference between catching a disease and missing it.



What You'll Learn Today

Building on your knowledge of confusion matrices, precision, recall, and F1 — we go deeper into **model evaluation beyond accuracy**.

Why Threshold Matters

How shifting the decision boundary changes predictions

Precision–Recall Trade-off

The fundamental tension in classification

PR & ROC Curves

Visualizing model performance across all thresholds

AUC

A single number to compare classifiers

Choosing Wisely

Matching your metric to your real-world application

Quick Review: The Confusion Matrix

Every binary classifier produces four outcomes. Make sure you can map each to a real-world scenario before we move on.

	Predicted: 0 (Healthy)	Predicted: 1 (Disease)
Actual: 0 (Healthy)	**TN** ✅	FP ⚠️
Actual: 1 (Disease)	FN ❌	**TP** ✅

TN — True Negative

Model predicts 0 (Healthy) → Patient is actually 0 (Healthy)

FP — False Positive

Model predicts 1 (Disease) → Patient is actually 0 (Healthy)

FN — False Negative

Model predicts 0 (Healthy) → Patient is actually 1 (Disease) 😳

TP — True Positive

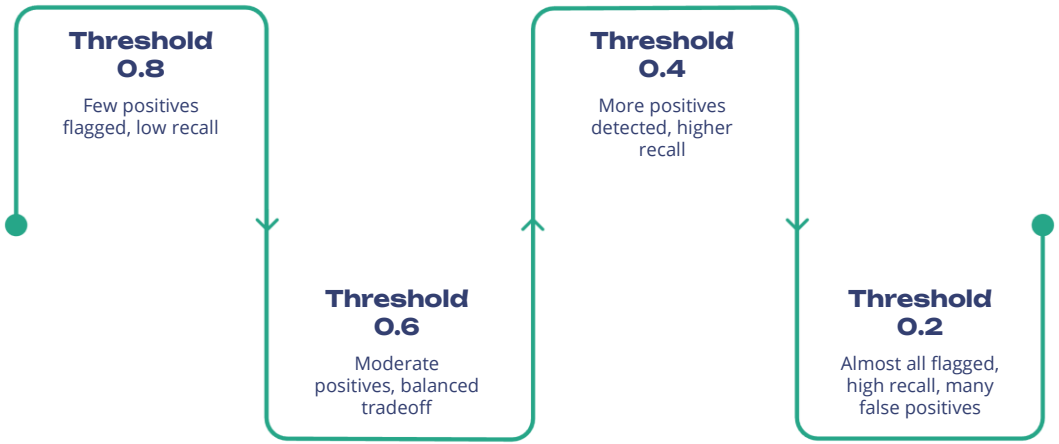
Model predicts 1 (Disease) → Patient is actually 1 (Disease)

Where Does the Threshold Come From?

Decision Trees and Random Forests don't output **labels** — they output **probabilities**. The threshold converts a probability into a Yes/No decision.

Patient	Probability	Prediction @ 0.5	Prediction @ 0.3
A	0.22	Healthy	Healthy
B	0.37	Healthy	**Disease ← changes!**
C	0.61	Disease	Disease

What Happens When the Threshold Changes?



The Core Insight

Lowering the threshold means the model flags more patients as Disease.

↑ Recall increases

More real patients are caught

↑ False Positives increase

More healthy patients are misclassified

↓ Precision decreases

Model's positive predictions become less reliable

Precision vs. Recall — The Intuition

🎯 Precision

"When the model says Disease — how often is it actually right?"

Out of all predicted positives, what fraction were truly positive?

🔍 Recall

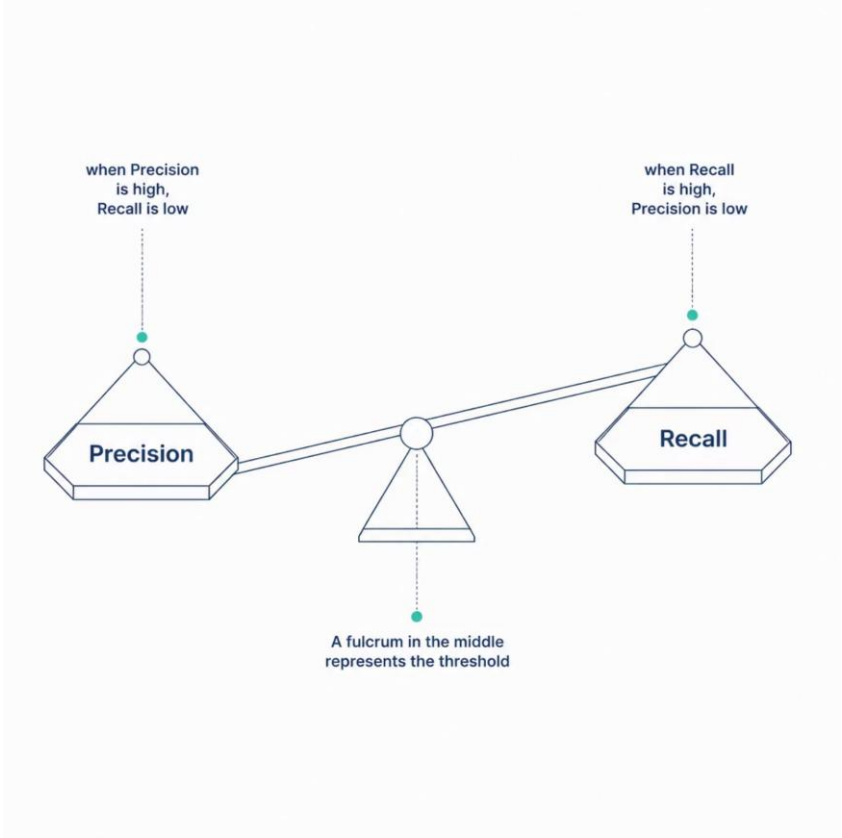
"Of all real disease patients — how many did the model catch?"

Out of all actual positives, what fraction did we detect?

A model can have high Recall by flagging *everyone* as Disease — but Precision would collapse. Neither alone tells the full story.



The Precision–Recall Trade-off



It depends on what matters most

Cancer Screening
Maximize Recall — missing a case is dangerous

Spam Detection
Maximize Precision — blocking real emails is costly

Airport Security
High Recall — flag anything suspicious

Our Running Example: Heart Failure Prediction

The Dataset

Clinical records of patients — age, blood pressure, ejection fraction, serum creatinine, and more.

The Model

A Decision Tree and a Random Forest trained in Python with `scikit-learn` to predict heart failure.

The Output

Each patient gets a probability score — not just a binary label. That score is what we'll analyze.

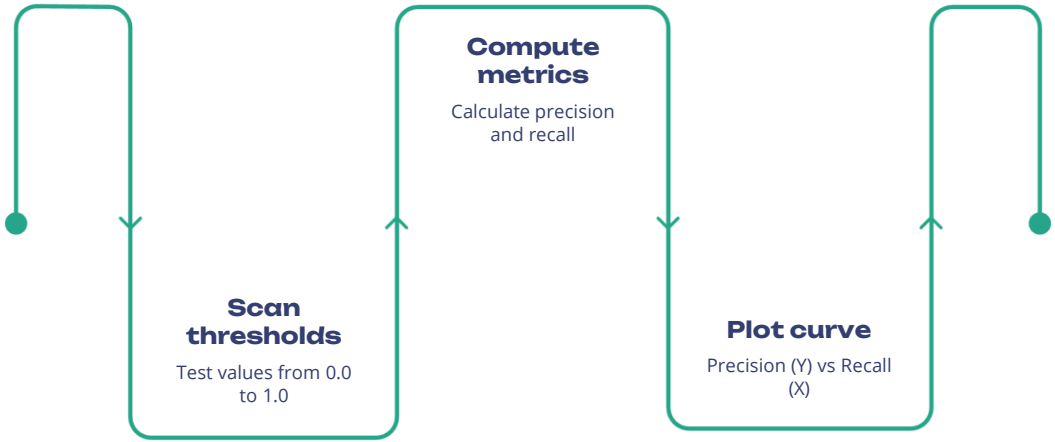


Threshold Experiment: Watching the Numbers Shift

We run the same model at five different thresholds. Notice how **every row tells a different story** about model behavior.

Threshold	Precision	Recall	Tradeoff
0.8	**Very High**	Very Low	Very conservative — misses many patients
0.6	High	Medium	Cautious but missing some cases
0.5	Balanced	Balanced	Default — often a reasonable start
0.3	Lower	**High**	Catches more — but more false alarms
0.1	Very Low	**Very High**	Flags almost everyone — unreliable

The Precision–Recall Curve



Why one threshold isn't enough

Instead of committing to a single threshold, we evaluate the model across **all possible values** at once.

X-axis → Recall

How many real positives did we catch?

Y-axis → Precision

How reliable were our positive predictions?

Top-right corner = ideal

High Precision AND high Recall — the dream model

Code Example: Scanning Thresholds for the Trade-off

```
step_factor = 0.05
threshold_value = 0.05
predicted_proba = best_rfc.predict_proba(test_X)  # Probability of
prediction

while threshold_value <= 1:  # continue to check best threshold upto
probability 0.8
    temp_thresh = threshold_value
    predicted = (predicted_proba[:, 1] >= temp_thresh).astype('int')  #
change the class boundary for prediction
    print("Threshold", threshold_value, "Precision",
precision_score(test_y, predicted), "\tRecall", recall_score(test_y,
predicted))
    threshold_value = threshold_value + step_factor
```

Low Threshold (0.05)

Recall = 1.0, Precision ≈ 0.60. Catches every disease case but many false alarms.

Mid Threshold (0.50)

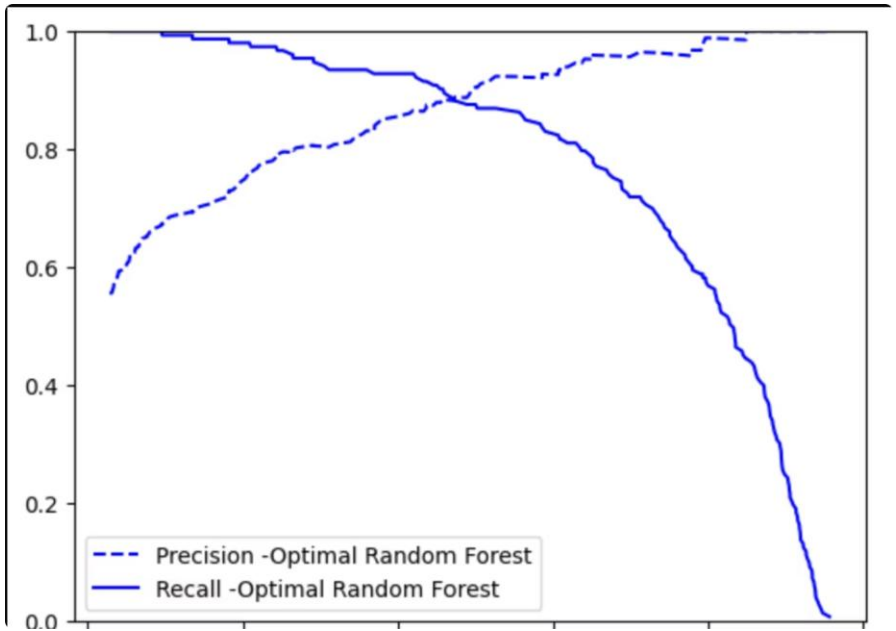
Precision ≈ 0.905, Recall ≈ 0.876. Best balance between the two metrics.

High Threshold (0.85)

Precision = 1.0, Recall ≈ 0.444. Very precise but misses over half of disease cases.

As threshold increases → Precision rises, Recall falls. This is the trade-off visualized in the Precision–Recall Curve.

Precision–Recall Curve: Optimal Random Forest



Recall starts high (1.0)

At low thresholds, the model catches all disease cases. Recall = 1.0, but Precision is only ~0.58.

Curves cross at ~0.88

Around threshold 0.50, Precision and Recall intersect near 0.88 — this is the sweet spot for balanced performance.

Precision rises, Recall drops

As threshold increases, Precision climbs toward 1.0 while Recall collapses to near 0.0 at high thresholds.

The crossing point of the two curves is a strong candidate for the optimal threshold — maximizing both Precision and Recall simultaneously.

Choosing a Threshold: It Depends on the Stakes

There is **no universally correct threshold**. The right choice depends on what kind of mistake you can afford.



🏥 Medical Diagnosis

Lower threshold — **maximize Recall**. Missing a disease is far worse than a false alarm.



💳 Fraud Detection

Balance both — too many false positives frustrate real customers.



🚙 Autonomous Driving

Maximize Recall for hazards — false positives just slow the car down.



📧 Email Spam

Favor Precision — blocking a real email is a worse error than missing spam.

The ROC Curve: A Different Lens

True Positive Rate (TPR)

= Recall. Of all actual positives, how many did we catch?
"Sensitivity"

False Positive Rate (FPR)

Of all actual negatives, how many did we *wrongly* flag?
"1 − Specificity"

Perfect Model

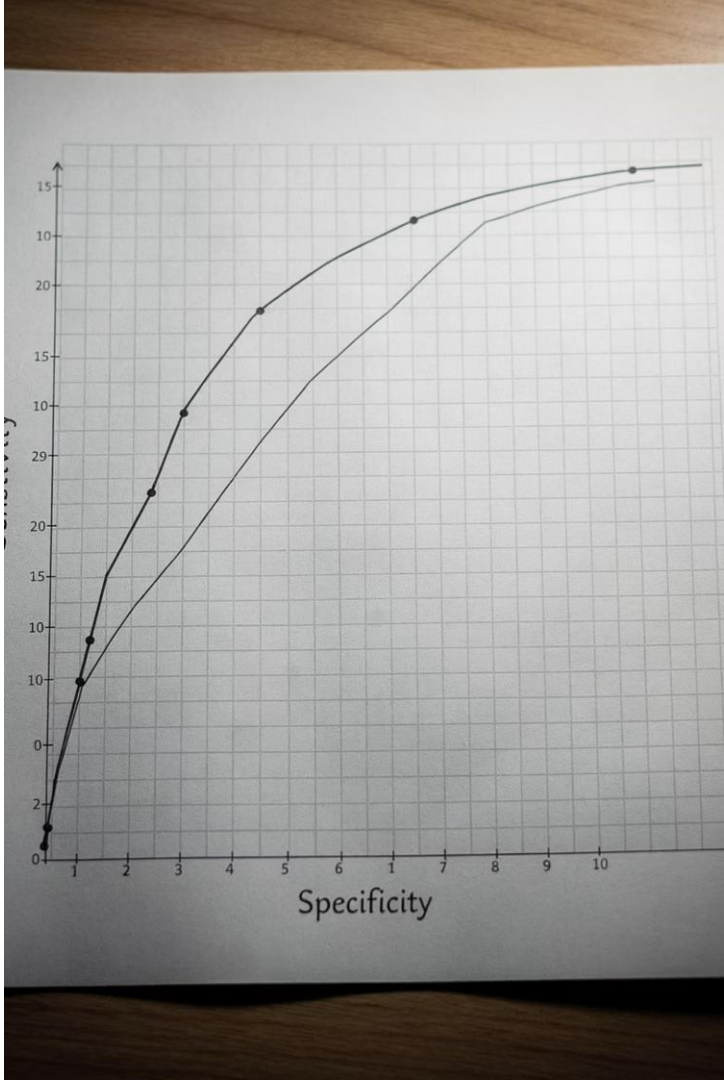
Hits the top-left corner: TPR = 1, FPR = 0

Random Guess

Falls on the diagonal line: TPR = FPR

Real Model

Somewhere between — closer to top-left is better



Building the ROC Curve — Step by Step



Each threshold produces one point on the curve. As we lower the threshold, **TPR rises** but so does **FPR**. The curve traces that path from (0,0) to (1,1).

Code Example: Computing FPR & TPR for the ROC Curve

```python
# prompt: calculate false positive rate and true positive rate of  best_rfc classifier for threshold 0.3

# Get the predicted probabilities for the positive class (class 1)
y_prob = best_rfc.predict_proba(test_X)[:, 1]

# Define the threshold
threshold = 0.4

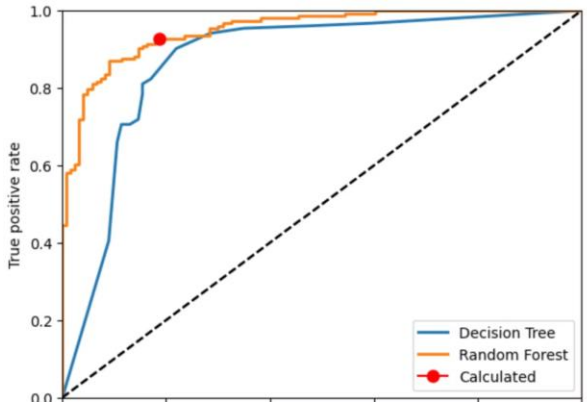
# Predict the classes based on the threshold
y_pred_threshold = (y_prob >= threshold).astype(int)

# Calculate the confusion matrix
cm = confusion_matrix(test_y, y_pred_threshold)

# Extract TN, FP, FN, TP from the confusion matrix
TN = cm[0, 0]
FP = cm[0, 1]
FN = cm[1, 0]
TP = cm[1, 1]

# Calculate False Positive Rate (FPR)
FPR = FP / (FP + TN)

# Calculate True Positive Rate (TPR)
TPR = TP / (TP + FN)
```



Step 1: Get Probabilities

`y_prob = best_rfc.predict_proba(test_X)[:, 1]` extracts the predicted probability of the positive class (Disease = 1) for each sample.

Step 2: Apply Threshold

`y_pred_threshold = (y_prob >= threshold).astype(int)` converts probabilities to binary predictions using `threshold = 0.4`.

Step 3: Extract from Confusion Matrix

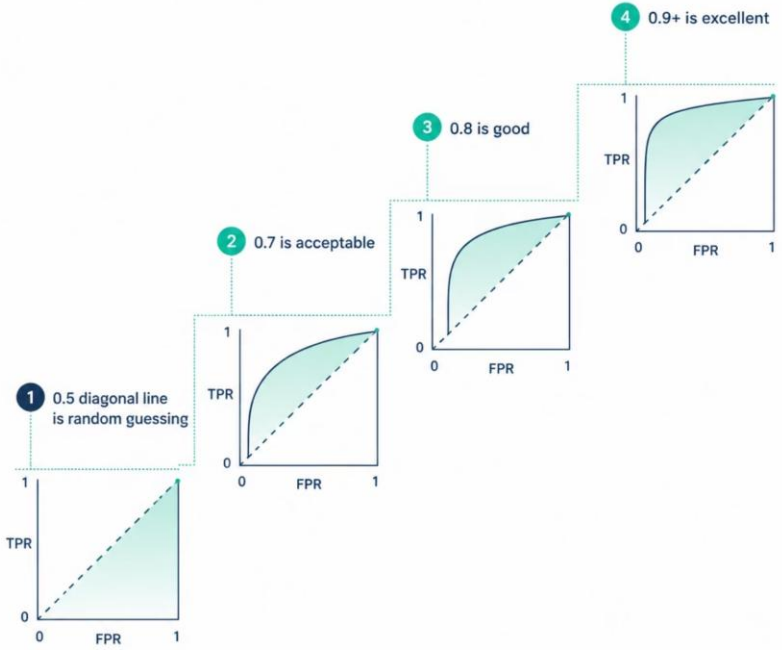
`TN = cm[0,0], FP = cm[0,1], FN = cm[1,0], TP = cm[1,1]` — maps each cell of the confusion matrix to its label.

Step 4: Compute FPR & TPR

`FPR = FP / (FP + TN)` — how many healthy patients were wrongly flagged. `TPR = TP / (TP + FN)` — how many disease patients were correctly caught.

Each threshold produces one (FPR, TPR) point. Repeating this across all thresholds traces the full ROC Curve.

Understanding AUC — One Number to Rule Them All



What does AUC tell us?

AUC = **Area Under the ROC Curve**. It measures how well a model separates positives from negatives across *all* thresholds.

AUC = 0.5	AUC = 0.7	AUC = 0.8	AUC ≥ 0.9

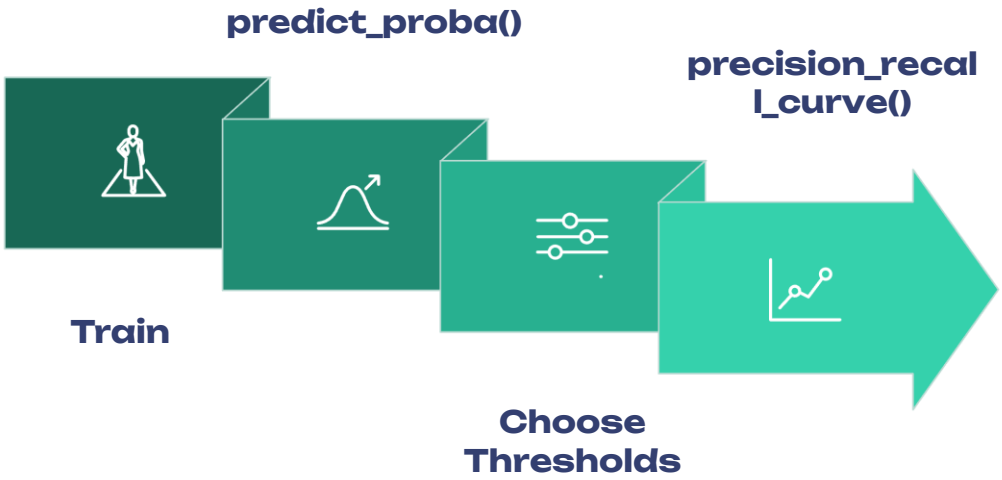
ROC vs. Precision–Recall: When to Use Which?

Both curves evaluate classifiers across thresholds — but they shine in **different scenarios**.

Dimension	ROC Curve	Precision–Recall Curve
Best for	Balanced class distributions	**Imbalanced datasets**
What it shows	TPR vs. FPR tradeoff	Precision vs. Recall tradeoff
Optimistic bias?	Yes — inflated on imbalanced data	No — more honest view
Typical use case	General model comparison	Fraud detection, rare disease

Heart failure datasets are often imbalanced — **Precision–Recall curves** are especially important here.

The Practical Python Workflow



Key scikit-learn functions

- `predict_proba()`

  Returns probability scores instead of hard labels

- `precision_recall_curve()`

  Computes P and R across all thresholds

- `roc_curve()`

  Computes TPR and FPR across all thresholds

- `roc_auc_score()`

  Returns a single AUC value for comparison

Real Medical Interpretation: Which Mistake Is Worse?

😨 False Negative

Missing a disease. A sick patient is sent home without treatment. Potentially fatal — this is what we want to minimize most.

⚠️ False Positive

False alarm. A healthy patient gets extra tests or treatment. Costly and stressful — but usually recoverable.

> **Discussion:** In heart failure prediction, which error carries higher risk? How does your answer change the threshold you'd choose?



Common Student Mistakes to Avoid

1

Using Accuracy Only

Accuracy hides poor performance on minority classes — especially dangerous in medical data.

2

Defaulting to Threshold = 0.5

The default is rarely optimal. Always ask: what does a false negative cost in this domain?

3

Ignoring Recall

A model with 95% Precision but 10% Recall is practically useless for screening.

4

Thinking AUC Picks the Threshold

AUC compares models — it does *not* tell you which threshold to deploy.

Summary: The Full Evaluation Pipeline



Every concept in this lecture connects back to a single idea: **probabilities give classifiers flexibility** — and that flexibility must be evaluated thoughtfully.



Thank You!

Go explore data — get your hands dirty, break things, and learn by doing.

📧 Email

MarcelFei@telhai.ac.il

🕐 Office Hours

Per Request

📍 Location

Building 6, Room 309, Eastern Campus